

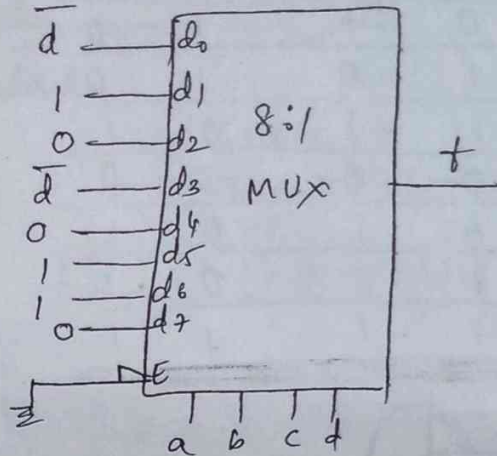
10) Implement the Boolean function with 8:1 MUX

$$f(A, B, C, D) = \sum m(0, 2, 6, 10, 12, 13) + d(3, 8, 14)$$

cd \ ab	00	01	11	10
00	1 ⁰	0 ¹	X ³	1 ²
01	0 ⁴	0 ⁵	0 ⁷	1 ⁶
11	1 ¹²	1 ¹³	0 ¹⁵	X ¹⁴
10	X ⁸	0 ⁹	0 ¹¹	1 ¹⁰

$$\begin{aligned} d_0 &= \bar{d} & d_4 &= 0 \\ d_1 &= 1 & d_5 &= 0 \\ d_2 &= 0 & d_6 &= 1 \\ d_3 &= \bar{d} & d_7 &= 0 \end{aligned}$$

Consider abc as select lines



Adders and Subtractors

→ Adding 2 single-bit binary values produces a sum and a carry output. This operation is called a half-adder and the ckt that realizes this function is called a half-adder.

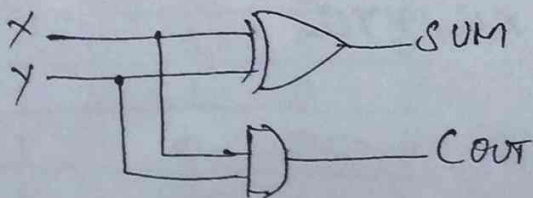
Inputs		Outputs	
x	y	S	Cont
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$$S = f(x, y) = \sum (1, 2)$$

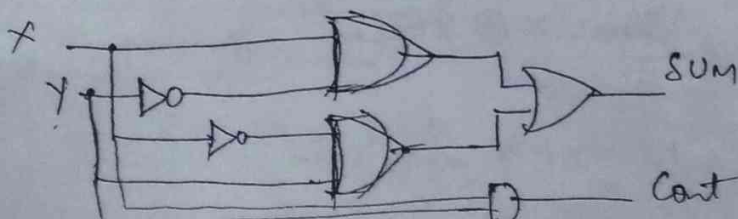
$$\text{Cont} = f(x, y) = \sum (3)$$

$$S = \bar{x}y + x\bar{y} = x \oplus y$$

$$\text{Cont} = xy$$



(or)



Adding 2 single-bit binary values with the inclusion of a carry input produces two outputs, a sum and a carry, this ckt is called a full-adder

X	Y	Cin	S	Cont
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

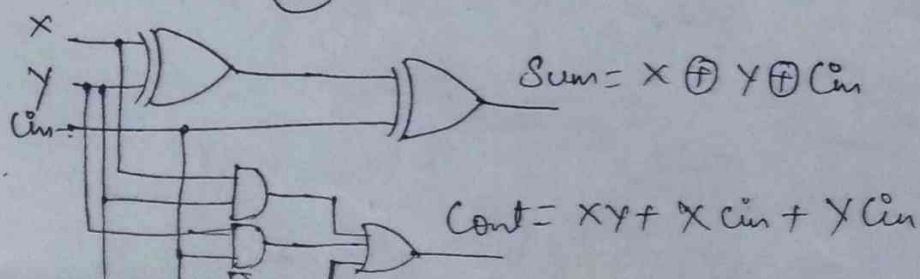
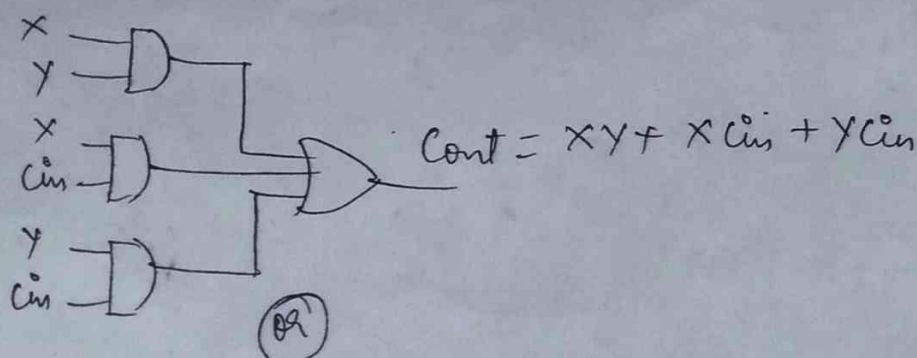
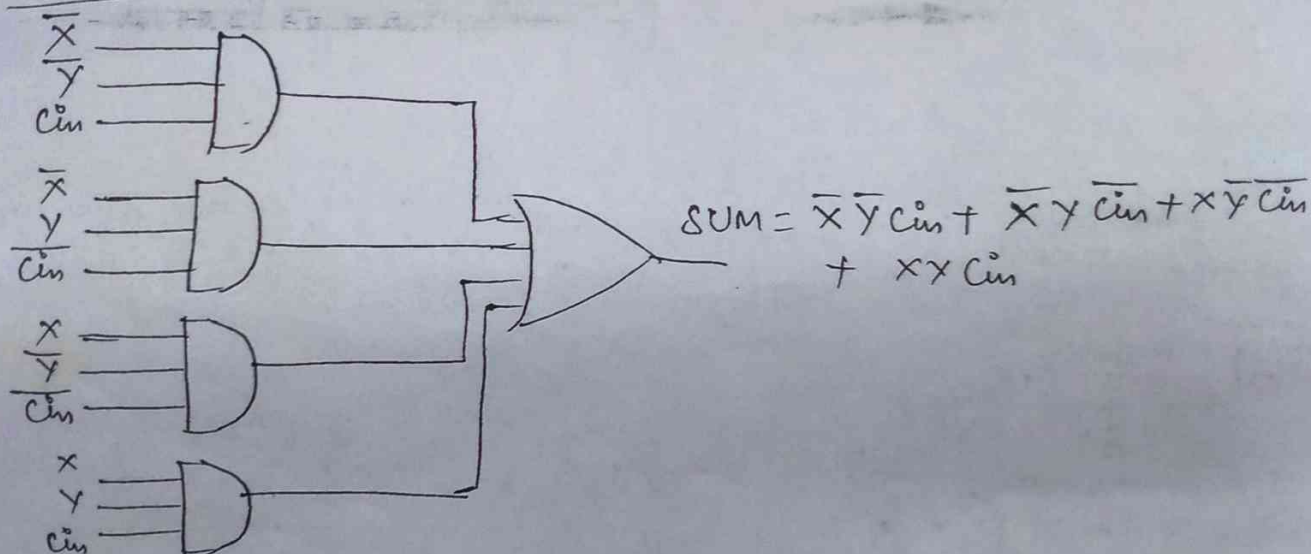
$$S = f(X, Y, C_{in}) = \Sigma(1, 2, 4, 7)$$

$$Cont = f(X, Y, C_{in}) = \Sigma(3, 5, 6, 7)$$

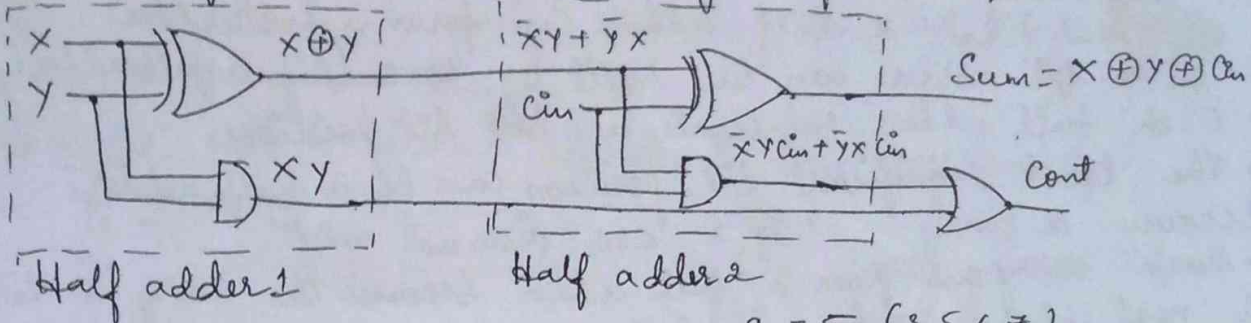
Use k-map for simplification

$$S = X \oplus Y \oplus C_{in}$$

$$Cont = XY + XC_{in} + YC_{in}$$



Two half adders and an OR gate forms a full-adder



$$Cont = \bar{x}y c_{in} + \bar{y}x c_{in} + xy$$

$$Cont = c_{in} (x \oplus y) + xy$$

(OR)

$$C = \sum (3, 5, 6, 7)$$

$$= \bar{x}y c_{in} + x \bar{y} c_{in} + xy c_{in} + xy c_{in}$$

$$C = c_{in} (x \oplus y) + xy$$

Binary Subtractors

Half Subtractor

X	Y	⊖	Bout
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

$$A = f(X, Y) = \sum (1, 2)$$

$$B = f(X, Y) = \sum (1)$$

$$A = \bar{x}y + x\bar{y} = x \oplus y$$

$$B = \bar{x}y$$

Full Subtractor

X	Y	B _{in}	⊖	Bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

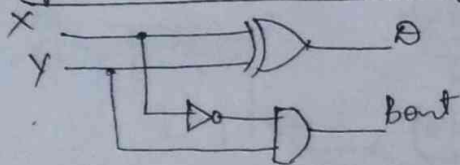
$$A = f(X, Y, B_{in}) = \sum (1, 2, 4, 7)$$

$$Bout = f(X, Y, B_{in}) = \sum (1, 2, 3, 7)$$

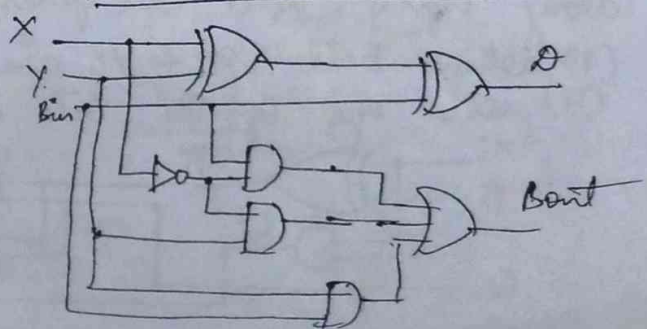
$$A = x \oplus y \oplus B_{in}$$

$$Bout = \bar{x}y + \bar{x}B_{in} + yB_{in}$$

Half Subtractor logic diagram

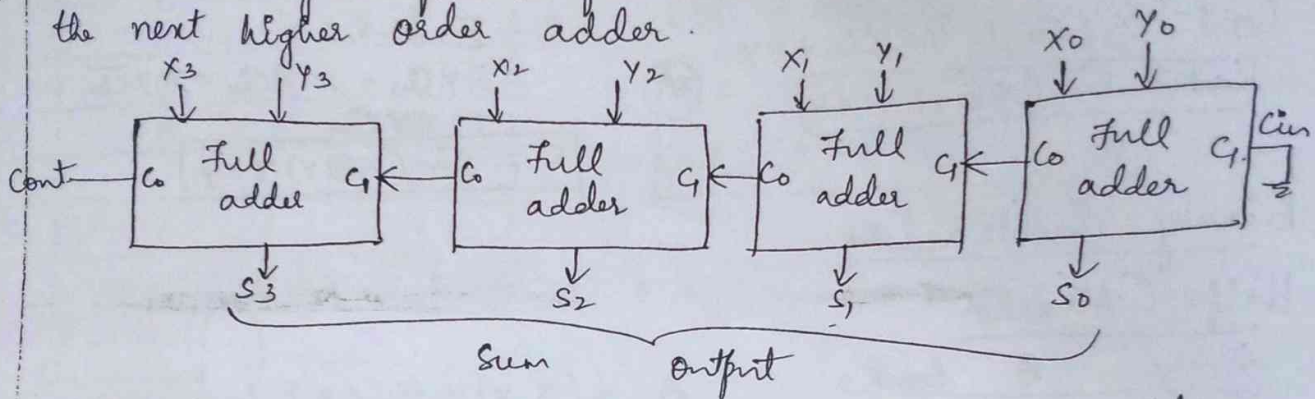


Full Subtractor logic diagram



Cascading Full adders

- We saw that 2 half-adders can form a full adder
- An n -bit adder can be built by cascading n full adders
- Each full adder represents a bit position
- The least significant bit position can be a half adder, because a carry in to a LSB does not exist.
- Each carry out from a full adder becomes the carry in to the next higher order adder.



- The carry input to the least significant full adder is grounded.
- The add time required is determined by calculating the propagation delay through the full adders.
- Carries propagate from one full adder to the next higher order full adder. This type of addition is called ripple-carry propagation.

$$\begin{array}{r} 0101 \\ + 0010 \\ \hline 1000 \end{array}$$

② Look-Ahead Carry

- Look-ahead carry or fast carry technique reduces propagation delay times through the adder.
- Look-ahead carry adder is based on the fact that a carry signal will be generated in 2 cases

(i) when both X_i & Y_i are equal to 1

(ii) when one of the bits is 1 and the carry in = 1.

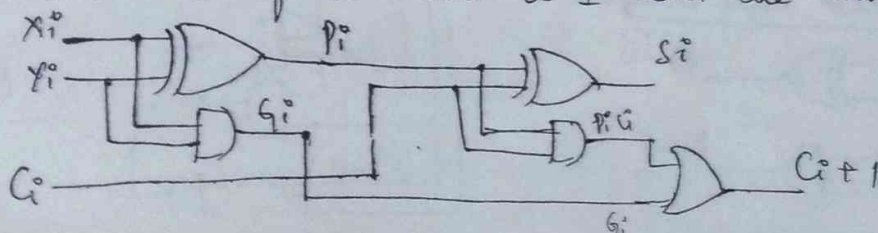


Fig. shows the full adder ckt used to add the operand bits in the i^{th} column namely X_i & Y_i & the carry bit coming from the previous column C_i .

In this ckt 2 internal signals P_i & G_i are given by

$$P_i = X_i \oplus Y_i \quad \text{--- (1)}$$

$$G_i = X_i Y_i \quad \text{--- (2)}$$

$$\therefore S_i = P_i \oplus C_i \quad \text{--- (3)}$$

$$C_{i+1} = G_i + P_i C_i \quad \text{--- (4)}$$

where,

G_i - carry generate signal

P_i - " Propagate "

Boolean expressions of the carry outputs of various stages

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 C_1 = G_1 + P_1 (G_0 + P_0 C_0) = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 C_2 = G_2 + P_2 (G_1 + P_1 G_0 + P_1 P_0 C_0)$$

$$= G_2 + P_2 G_1 + P_1 P_2 G_0 + P_0 P_1 P_2 C_0$$

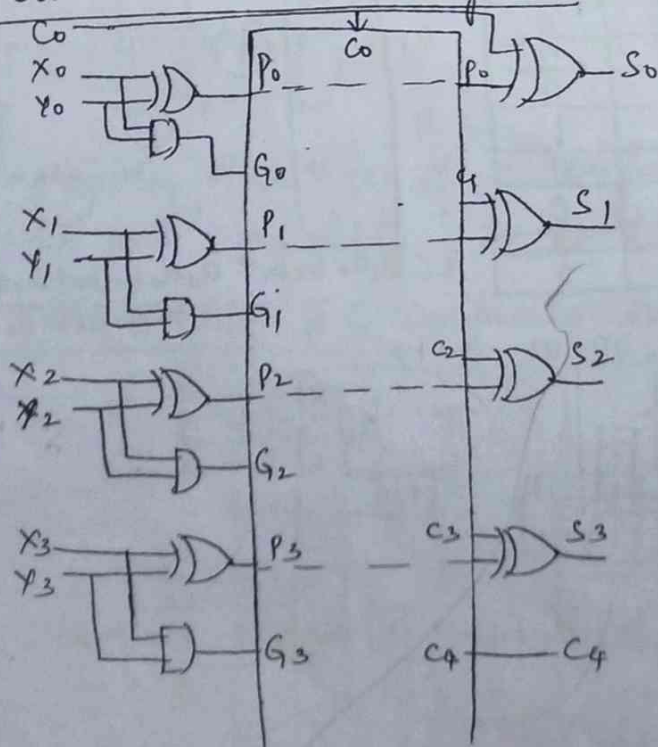
$$C_4 = G_3 + P_3 C_3 = G_3 + P_3 (G_2 + P_2 G_1 + P_1 P_2 G_0 + P_0 P_1 P_2 C_0)$$

$$= G_3 + P_3 G_2 + P_2 P_3 G_1 + P_1 P_2 P_3 G_0 + P_0 P_1 P_2 P_3 C_0$$

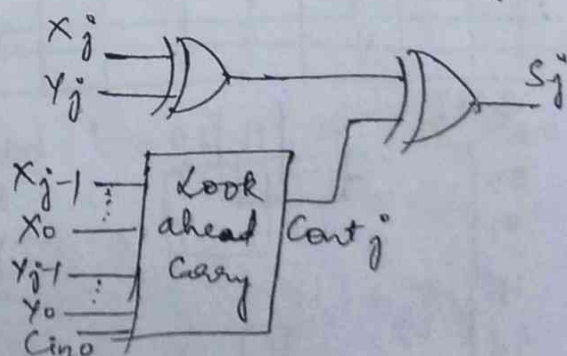
In general

$$C_{out j+1} = G_j + P_j C_{in j}$$

4 bit look ahead carry adder



Generation of C_{in} from a look ahead carry



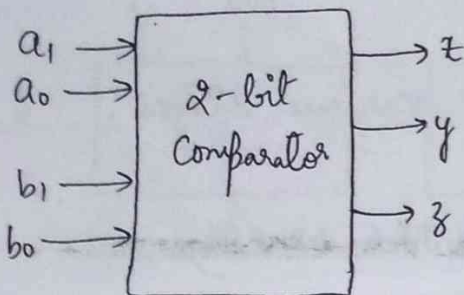
Binary Comparators

A binary magnitude comparator is a logic ckt that provides output information indicating the relative magnitude of two input operands.
 → 3 output conditions exist as a result of comparison of 2 operands:

$$* A = B$$

$$* A > B$$

$$* A < B$$



$$x = 1 \quad \text{if } a_1 a_0 = b_1 b_0$$

$$y = 1 \quad \text{if } a_1 a_0 > b_1 b_0$$

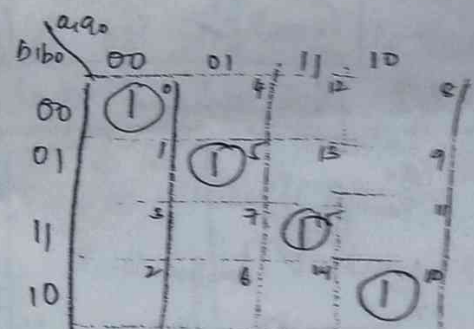
$$z = 1 \quad \text{if } a_1 a_0 < b_1 b_0$$

Inputs				Outputs		
A		B		$A=B$ (x)	$A>B$ (y)	$A<B$ (z)
a_1	a_0	b_1	b_0			
0	0	0	0	1	0	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	0	1
0	1	0	0	0	1	0
0	1	0	1	1	0	0
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	0	1	0
1	0	0	1	0	1	0
1	0	1	0	1	0	0
1	0	1	1	0	0	1
1	1	0	0	0	1	0
1	1	0	1	0	1	0
1	1	1	0	0	1	0
1	1	1	1	1	0	0

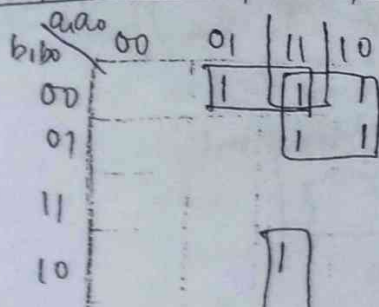
$$x = \Sigma (0, 5, 10, 15)$$

$$y = \Sigma (4, 8, 9, 12, 13, 14)$$

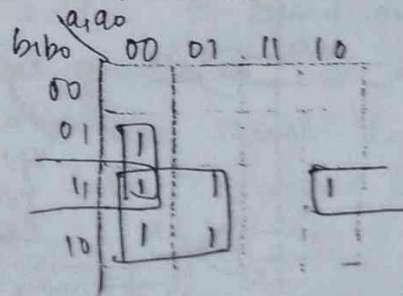
$$z = \Sigma (1, 2, 3, 6, 7, 11)$$



$$x = a_1 \bar{a}_0 \bar{b}_1 \bar{b}_0 + \bar{a}_1 a_0 \bar{b}_1 \bar{b}_0 + a_1 a_0 \bar{b}_1 \bar{b}_0 + \bar{a}_1 \bar{a}_0 b_1 b_0$$



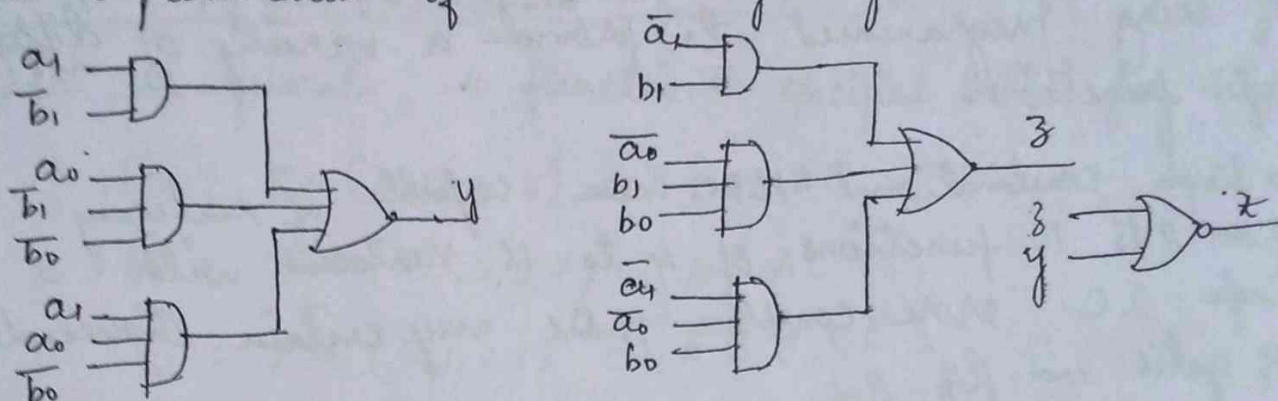
$$y = a_1 b_1 + a_0 \bar{b}_1 \bar{b}_0 + a_1 a_0 \bar{b}_0$$



$$z = \bar{a}_1 b_1 + \bar{a}_1 \bar{a}_0 b_0 + \bar{a}_0 b_1 b_0$$

* x can also be written as $x = \overline{y+z}$

The implementation of 2-bit binary comparator is shown below



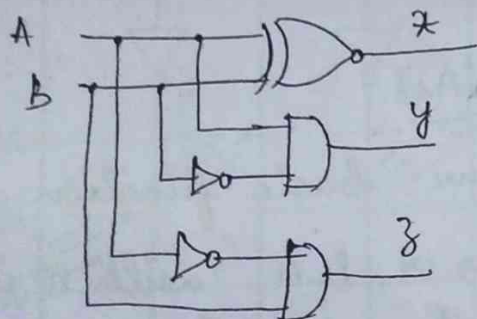
1 bit Comparator

A	B	x	y	z
0	0	1	0	0
0	1	0	0	1
1	0	0	1	0
1	1	1	0	0

$$x = \Sigma(0, 3) = \bar{a}\bar{b} + ab = \bar{a \oplus b}$$

$$y = \Sigma(2) = a\bar{b}$$

$$z = \Sigma(1) = \bar{a}b$$



→ Increasing the size of a comparator beyond 2 bits would produce an unwieldy truth table.

74LS85 (4-bit IC Comparator)

